

FASE 3.5

INFORME FORENSE

Busqueda Inteligente de Conversaciones

VitaZen - Mentor IA

Filtrado instantaneo por titulo en sidebar

Parametro	Valor
Proyecto	VitaZen
Componente	MentorChat.tsx
Fase	3.5 - Busqueda Inteligente de Conversaciones
Fecha	2026-07-15
Archivo modificado	src/components/mentor/MentorChat.tsx
Lineas afectadas	~35 lineas (7 puntos de edicion)
Paquetes nuevos	0 (ninguno)
Motor IA modificado	No
APIs modificadas	No
Prisma modificado	No

1. Auditoria Inicial

Se realizo una auditoria completa del sistema de conversaciones de VitaZen, enfocada en el sidebar, el listado de threads, las APIs existentes, el modelo Prisma AIThread, y el comportamiento responsive. El objetivo era determinar la solucion mas eficiente para implementar busqueda sin romper ninguna arquitectura.

1.1 Sidebar y listado de conversaciones

El sidebar (variable sidebarContent, linea 661) se renderiza tanto en el sidebar desktop (div con clase w-72, linea 1058) como en el drawer mobile (fixed panel, linea 1270). Esto significa que cualquier cambio en sidebarContent se aplica automaticamente a ambas vistas sin duplicacion de codigo. El listado de threads se agrupa por fecha (Hoy, Ayer, Esta semana, Este mes, Anterior) mediante el hook useMemo groupedThreads. Los tabs "Todas" y "Archivadas" filtran entre threads activos y archivados. No existia ningun mecanismo de busqueda previo.

1.2 API de threads y Prisma

El endpoint GET /api/ai/threads (route.ts, 184 lineas) devuelve todos los threads del usuario ordenados por updatedAt descendente. Los usuarios FREE tienen un limite de 10 threads (HISTORY_LIMIT_FREE = 10) y los PREMIUM son ilimitados hasta 100. El modelo AIThread en Prisma tiene un campo title (String) con indice compuesto [userId, archived] pero sin indice de texto. La API soporta el parametro ?archived pero no tiene parametro de busqueda.

1.3 Volumen de datos

El volumen maximo de threads por usuario es: FREE = 10 threads, PREMIUM = 100 threads. Cada thread pesa aproximadamente 200 bytes en memoria (id, title, archived, updatedAt, createdAt). El total maximo en memoria es de 20 KB para FREE y 20 KB para PREMIUM. Este volumen es lo suficientemente pequeno para que un filtrado en memoria con String.prototype.includes() sea instantaneo (menos de 0.1ms para 100 threads), lo que elimina la necesidad de busqueda en servidor.

2. Arquitectura Encontrada

El flujo de datos del sidebar sigue una cadena de derivacion de estado que va desde los threads originales hasta los threads agrupados que se renderizan en la UI:

Variable	Tipo	Descripcion
threads	Estado original (useState)	Todos los threads del usuario
activeThreads	useMemo(filter)	threads.filter(t => !t.archived)
archivedThreads	useMemo(filter)	threads.filter(t => t.archived)
visibleThreads	useMemo(select)	tab activo ? active : archived
groupedThreads	useMemo(reduce)	Agrupados por fecha (DATE_GROUP_ORDER)

La busqueda se inserta entre visibleThreads y groupedThreads, anadiendo un nuevo eslabon: searchedThreads. Esta posicion es la mas eficiente porque filtra sobre el conjunto ya reducido por el tab (activos o archivados), y el resultado fluye naturalmente hacia la agrupacion por fecha y los estados vacios.

3. Solucion Implementada

La solucion es 100% en cliente, sin modificar la API ni Prisma. Se basa en 7 puntos de edicion en un unico archivo, mas 1 import nuevo de lucide-react.

3.1 Import Search de lucide-react

Se anadio el icono Search a los imports de lucide-react (linea 37). Este icono se utiliza tanto en el campo de busqueda como en el estado vacio de resultados. Lucide-react ya estaba instalado como dependencia del proyecto, por lo que no se anadio ningun paquete nuevo.

3.2 Estado searchQuery

Se anadio un nuevo estado: `const [searchQuery, setSearchQuery] = useState("")` en la linea 160, junto a los estados existentes del sidebar (`tab`, `drawerOpen`). Este estado almacena el texto de busqueda actual. Se inicializa como cadena vacia, lo que significa que la busqueda esta inactiva por defecto y no afecta el renderizado hasta que el usuario escriba algo.

3.3 Filtrado searchedThreads

Se anadio un nuevo `useMemo` entre `visibleThreads` y `groupedThreads` que aplica el filtro de busqueda:

```
const searchedThreads = useMemo(() => {  
  <br/><br/>const q =  
  searchQuery.trim().toLowerCase();<br/><br/>if (!q) return  
  visibleThreads;<br/><br/>return visibleThreads.filter(t =>  
  t.title.toLowerCase().includes(q));<br/>}, [visibleThreads, searchQuery]);
```

El algoritmo `trim()` elimina espacios en los extremos, `toLowerCase()` convierte tanto el query como el titulo a minusculas para busqueda case-insensitive, e `includes()` realiza una busqueda parcial (subcadena). Cuando el query esta vacio (solo espacios o cadena vacia), retorna `visibleThreads` sin filtrar, lo que evita recalcular cuando no hay busqueda activa.

3.4 Campo de busqueda en sidebarContent

Se inserto un campo de busqueda entre el tab selector y la lista de threads (linea 730). El diseno utiliza los mismos patrones visuales del sidebar: fondo `bg-[#111]`, borde `border-[#1a1a1a]`, texto `text-xs` y `text-white`, y el champagne como color de focus. El icono Search se posiciona de forma absoluta a la izquierda como hint visual. Cuando hay texto de busqueda, aparece un boton X a la derecha para limpiar la busqueda con un solo toque, utilizando el icono X ya importado de lucide-react.

3.5 Estado vacio de busqueda

Se anadio un nuevo estado vacio (linea 900) que se muestra cuando hay texto de busqueda y no hay resultados. Muestra el icono Search, el texto "Sin resultados" y el query buscado entre comillas para contexto. Los estados vacios existentes ("Sin conversaciones" y "Sin archivadas") se modificaron para ocultarse cuando hay busqueda activa (se anadio la condicion `&& !searchQuery`), evitando asi que el usuario vea "Sin conversaciones" cuando en realidad tiene conversaciones pero ninguna coincide con su busqueda.

4. Archivos Modificados

#	Ubicacion	Tipo	Descripcion
1	Linea 37	Import	Search anadido a lucide-react imports

#	Ubicacion	Tipo	Descripcion
2	Linea 160	Estado	searchQuery (useState)
3	Lineas 633-637	useMemo	searchedThreads (filtro por titulo)
4	Linea 640	useMemo	groupedThreads ahora usa searchedThreads
5	Lineas 730-752	JSX	Campo de busqueda con icono y boton limpiar
6	Lineas 879, 890	JSX	Estados vacios ocultos durante busqueda
7	Lineas 900-909	JSX	Estado vacio de busqueda (sin resultados)

Total: 1 archivo modificado, 7 puntos de edicion, ~35 lineas de codigo. 0 archivos nuevos, 0 dependencias instaladas, 0 APIs modificadas, 0 cambios en Prisma.

5. Justificacion Tecnica

5.1 Por que busqueda en cliente y no en servidor

La decision de implementar la busqueda 100% en el cliente se basa en tres factores cuantitativos. Primero, el volumen de datos es minimo: maximo 100 threads por usuario, cada uno con un titulo de aproximadamente 50-100 caracteres. Un filter() con includes() sobre 100 strings toma menos de 0.1ms en cualquier navegador moderno. Segundo, la latencia de una busqueda en servidor (roundtrip HTTP + query Prisma + respuesta JSON) seria de 50-200ms, lo que anadiria un loader visible y contradeciria el requisito de "busqueda instantanea sin loaders". Tercero, los threads ya estan cargados en memoria como estado de React, por lo que no hay costo de red adicional.

5.2 Por que no debounce

Un debounce (retrasar la busqueda Nms despues de la ultima pulsacion) es util cuando el filtrado tiene un costo significativo (llamadas API, procesamiento pesado). En este caso, el filtrado es una operacion O(n) con n maximo 100, que se ejecuta en microsegundos. Anadir un debounce de 200-300ms empeoraria la experiencia percibida porque el usuario veria un retraso innecesario entre escribir y ver resultados. La actualizacion instantanea (cada pulsacion de tecla) es la mejor experiencia.

5.3 Por que useMemo y no filtrado en el render

El hook useMemo con las dependencias [visibleThreads, searchQuery] garantiza que el filtrado solo se re-ejecute cuando cambian los threads visibles o el texto de busqueda. Sin memoizacion, el filtrado se ejecutaria en cada render del componente (por ejemplo, cuando cambia un mensaje en el chat, cuando el scroll se mueve, o cuando se actualiza el contador de mensajes restantes). Con useMemo, estos renders no disparan el filtro, ahorrando calculos innecesarios.

6. Compatibilidad con Toda la Arquitectura Existente

Componente	Impacto
11-layer pipeline	No afectado. Opera sobre mensajes, no sobre el sidebar.
Auth / Firebase	No afectado. No se toco autenticacion.
Neon / PostgreSQL	No afectado. No se modifiko la base de datos.
Prisma schema	No afectado. No se modifiko el esquema.

Componente	Impacto
Groq API	No afectado. La busqueda es local al cliente.
GET /api/ai/threads	No afectado. Se mantiene sin cambios.
Premium / FREE	No afectado. Los tabs y gates funcionan igual.
Markdown (FASE 3.2)	No afectado. Renderiza respuestas assistant.
Copiar respuesta (FASE 3.3)	No afectado. Opera sobre mensajes.
Input multilinea (FASE 3.4)	No afectado. Opera sobre el compositor.
Scroll automatico	No afectado. El scroll del chat es independiente.
Drawer mobile	Compatible. sidebarContent se usa en ambos.
Sidebar desktop	Compatible. sidebarContent se usa en ambos.

7. Rendimiento

7.1 Costo por pulsacion de tecla

Cada pulsacion de tecla en el campo de busqueda ejecuta: (1) setSearchQuery(), que es una actualizacion de estado React $O(1)$; (2) re-evaluacion del useMemo searchedThreads, que ejecuta trim() + toLowerCase() + filter() sobre maximo 100 strings; (3) re-evaluacion del useMemo groupedThreads, que ejecuta reduce() sobre el resultado filtrado. El costo total por pulsacion es $O(n)$ donde n es el numero de threads visibles (maximo 100). En practica, esto toma menos de 0.1ms en cualquier dispositivo moderno, incluyendo iPhones de 3 generaciones atras.

7.2 Memoria

La busqueda no anade ninguna estructura de datos persistente. El estado searchQuery es una string (8 bytes de overhead + longitud del texto). El useMemo searchedThreads crea un nuevo array de referencias (no copia los objetos thread), con un overhead de aproximadamente 8 bytes por referencia. Para 100 threads, esto es 800 bytes adicionales de memoria temporal durante el filtrado, que es despreciable.

7.3 Impacto en renders

Cuando searchQuery cambia, se re-renderiza el componente MentorChat. Sin embargo, este re-render ya ocurría con cada pulsacion de tecla en cualquier input del componente (por ejemplo, el textarea del compositor desde FASE 3.4). La diferencia es que ahora se recalculan searchedThreads y groupedThreads, pero ambos son useMemo con dependencias correctas, lo que significa que se saltan si las dependencias no cambiaron.

8. Riesgos Detectados

8.1 Riesgo: Titulo con caracteres especiales

Nivel: Nulo. El metodo toLowerCase() e includes() de JavaScript manejan correctamente todos los caracteres Unicode incluyendo acentos espanoles (a, e, i, o, u, n), simbolos y emojis. La busqueda "ansiedad" encontrara "Ansiedad", "ANSIEDAD" y "ansiedad" por igual. No se requiere normalizacion Unicode adicional.

8.2 Riesgo: Saturacion del sidebar en busquedas amplias

Nivel: Bajo. Si el usuario escribe una sola letra comun (como "a" o "e"), la busqueda devolvera la mayoria de los threads. Esto no causa ningun problema funcional ni de rendimiento porque el listado ya soporta mostrar todos los threads del usuario con scroll interno. La experiencia es identica a no tener busqueda activa.

8.3 Riesgo: Busqueda persistente al cambiar de conversacion

Nivel: Bajo (comportamiento intencional). Cuando el usuario busca y selecciona un thread, el query de busqueda permanece activo. Esto es consistente con el comportamiento de aplicaciones como iMessage y Slack, donde el filtro persiste hasta que el usuario lo limpia explicitamente con el boton X o borra el texto. Si se desea limpiar automaticamente al seleccionar un thread, se puede anadir setSearchQuery("") en el handler onClick del thread, pero esto esta fuera del alcance de esta FASE.

9. Validaciones Realizadas

Validacion	Resultado
TypeScript (tsc --noEmit)	0 errores nuevos en MentorChat.tsx
ESLint (eslint MentorChat.tsx)	0 errores, 0 warnings
Compilacion Turbopack	Compilacion exitosa en 24.6s
Import Search (lucide-react)	Correcto, icono disponible
Estado searchQuery	Inicializado vacio, tipo string
Filtrado searchedThreads	useMemo con deps correctas
groupedThreads actualizado	Usa searchedThreads en vez de visibleThreads
Campo de busqueda UI	Renderiza en sidebar y drawer (sidebarContent)
Boton limpiar (X)	Solo visible cuando searchQuery no esta vacio
Estado vacio "Sin resultados"	Solo visible cuando hay query y 0 resultados
Estados vacios originales	Ocultos cuando searchQuery esta activo
aria-label campo busqueda	"Buscar conversaciones por titulo"
aria-label boton limpiar	"Limpiar busqueda"
Tabs y Premium gates	No afectados por la busqueda

10. Posibles Mejoras Futuras

- Limpiar busqueda automaticamente al seleccionar un thread resultado.
- Limpiar busqueda al cambiar de tab (Todas/Archivadas).
- Anadir atajo de teclado (Ctrl+K / Cmd+K) para foco rapido en el campo.
- Resaltar el texto coincidente en los titulos de los threads encontrados.
- Extender la busqueda al contenido de los ultimos mensajes (requiere API).
- Implementar busqueda semantica con embeddings para coincidencias conceptuales.
- Anadir parametro ?q= al GET /api/ai/threads para busqueda server-side si el numero de threads por usuario crece significativamente en el futuro.
- Persistir el query de busqueda en localStorage para mantenerlo entre sesiones.

11. Recomendaciones

La implementacion actual es optima para el volumen de datos existente (maximo 100 threads por usuario). Sin embargo, si en el futuro el limite de threads PREMIUM aumenta significativamente (por ejemplo, a 1000+), se recomienda migrar a una busqueda server-side con parametro `?q=` en el endpoint GET `/api/ai/threads`. Esto permitiria a Prisma aplicar el filtro en la consulta SQL (`WHERE title ILIKE %q%`) aprovechando el motor de base de datos y reduciendo la cantidad de datos transferidos al cliente. Para esto, se anadiria un indice de texto en Prisma sobre el campo `title` del modelo `AIThread`, y el parametro `q` se anadiria al `searchParams` del GET handler.

Se recomienda tambien considerar la limpieza automatica del campo de busqueda cuando el usuario cambia de tab o selecciona un thread, ya que esto mejora la experiencia al evitar que el usuario vea un sidebar vacio despues de navegar. Esta mejora es menor y puede implementarse en una FASE posterior sin riesgo de regresion.

12. Resumen Ejecutivo

La FASE 3.5 implementa la busqueda inteligente de conversaciones en el sidebar del Mentor IA. La solucion es 100% en cliente (sin modificar API ni Prisma), instantanea (sin loaders, debounce ni parpadeos), y se integra de forma transparente tanto en el sidebar desktop como en el drawer mobile a traves del contenido compartido `sidebarContent`.

La busqueda filtra por titulo de forma case-insensitive y parcial, se actualiza con cada pulsacion de tecla, incluye un boton para limpiar, y muestra un estado vacio dedicado cuando no hay resultados. Los estados vacios originales ("Sin conversaciones" y "Sin archivadas") se ocultan inteligentemente durante la busqueda activa.

Un unico archivo fue modificado (`MentorChat.tsx`) en 7 puntos de edicion (~35 lineas), mas 1 import nuevo (`Search` de `lucide-react`). 0 dependencias nuevas, 0 APIs tocadas, 0 cambios en Prisma, 0 motores del Mentor IA modificados. La validacion arroja 0 errores TypeScript nuevos, 0 warnings ESLint, y compilacion exitosa. La funcionalidad se percibe como una caracteristica nativa de VitaZen.